JOIN OUR WHATSAPP GROUP FOR UPDATEDS

WHATSAPP GROUP

VISIT OUR WEBSITE FOR STUDY MATERIALS

WEBSITE

PBCST 404 COMPUTER ORGANIZATION AND ARCHITECTURE

(Common to CS/CD/CR/CA/AD/CB/CN/CC/CU/CG)

SEMESTER S4

MODULE 1

Prasija J

Downloaded from KTUNOTES.LIVE

Basic Structure / Functional Unit of a Computer

The computer system is divided into five separate units for its operation as shown in figure 1.

- **Input & Output Unit** : Input devices read data from a source such as magnetic disks and translate the data into electronic impulses [ADC] for transfer into CPU. Whereas output devices convert electronic impulses [DAC] into human readable form.
- **Memory Unit (MU)** : It is a collection of storage cells along with associated circuits and used for storing data and instructions. It is the basic and fundamental components of a computer system.

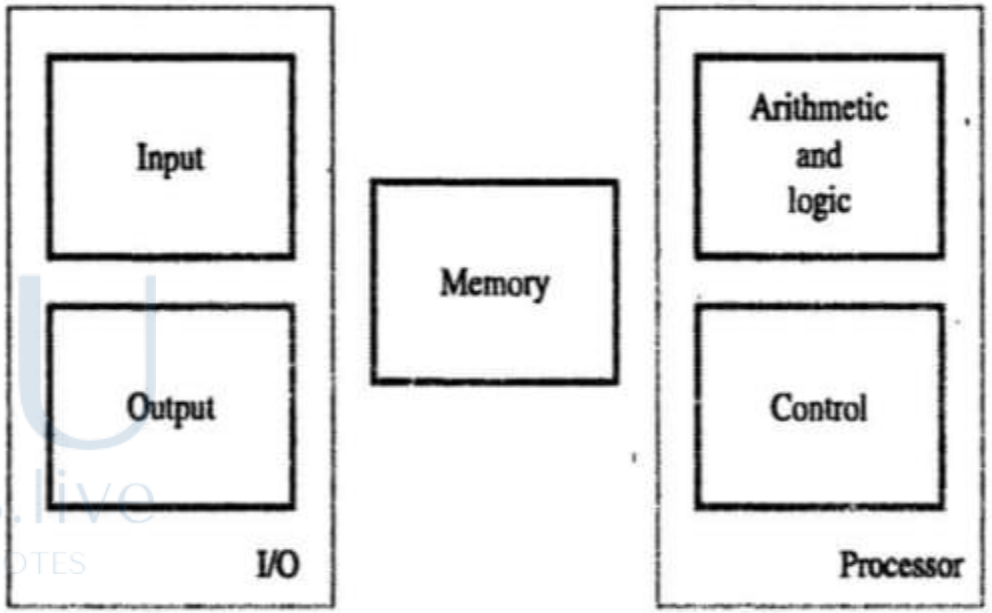


Figure 1

Types of Data Storage: Primary - RAM (main memory)

Secondary – Hard disk, Pendrive and Optical media

Downloaded from KTUNOTES.LIVE

- **Arithmetic Logic Unit (ALU)**:
  - ✓ It is responsible for actual processing of data and instructions.
  - ✓ The data is transferred from memory to ALU
  - ✓ It performs arithmetic (Add, sub, Mul, div), Logical (AND, OR, NOT) and comparison operations (>, <, =).
  - ✓ After processing the output is sent back to the memory unit and data is stored for further processing.
- **Control Unit (CU)** : It controls and coordinates all operations of the computer.
  - ✓ It determines the sequence of execution of instructions.
  - ✓ Fetches instructions from main memory, interprets and decodes the instruction.
  - ✓ Generates control signals for other units to execute operations.
  - ✓ Manages processing of programs stored in memory and acts as the manager of all system operations.
- **Central Processing Unit (CPU)** : It consists of ALU, CU, MU
  - ✓ It is the brain of the computer.
  - ✓ It performs all calculations, makes logical decisions, control and directs other components.
  - ✓ It activates and manages operations of all parts of the computer system.

KTU ktunotes.live 2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

Memory Map

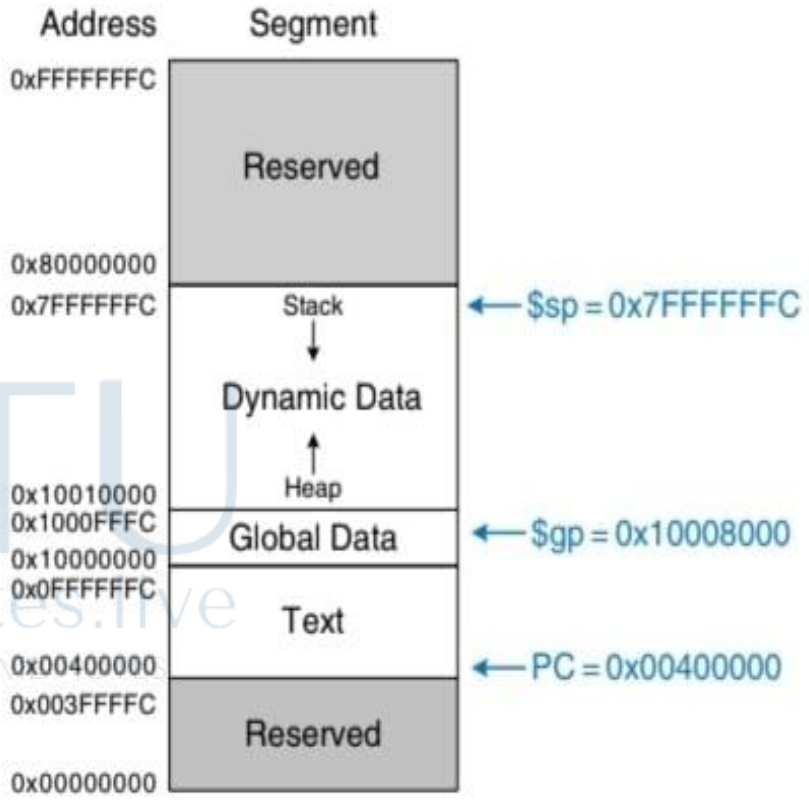
- With 32-bit addresses, the NIPS address space spans 2^{32} bytes = 4 gigabytes(GB).
- Word addresses are divisible by 4 and ranges from 0 to 0xFFFFFFFC.
- The MIPS architecture divides the address space into four parts or segments:

✓ Text Segment

✓ Global Data Segment

✓ Dynamic Data Segment

✓ Reserved Segment



Downloaded from KTUNOTES.LIVE

- **Text Segment:**
- ✓ Stores the machine language instructions of the program.
- ✓ Size is approximately 256 MB
- ✓ Begins at address 0x00400000
- ✓ PC is initialized to 0x00400000
- ✓ The four MSB are 0, allowing the j (jump) instruction to jump anywhere within the text segment directly.
- ✓ Contains only executable code and is generally read-only

- **Global Data Segment :**
- ✓ Stores global variables that are accessible by all procedures.
- ✓ Global variables are defined at program start-up, before execution begins.
- ✓ In C, these are variables declared outside the main() function.
- ✓ Size of the global data segment is 64 KB
- ✓ Starts at address 0x10000000
- ✓ Accessed using global Pointer register $gp where it is initialized to 0x10008000.
- ✓ $gp remains constant during program execution.
- ✓ Global variables are accessed using base addressing mode with 16-bit signed offsets from $gp

KTU
ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

- **Dynamic data Segment:**
  - ✓ Contains both stack and the heap.
  - ✓ Data is allocated and deallocated at runtime.
  - ✓ This is the largest memory segment, spanning nearly 2 GB
  - ✓ Address ranges approximately from 0x10010000 to 0x7FFFFFFC

Stack:

- ✓ Used for saving and restoring registers, storing local variables, supporting procedure calls.
- ✓ Operates in Last-in-First-Out (LIFO) order.
- ✓ Grows downward in memory and stack pointer $sp is initialized to 0x7FFFFFFC
- ✓ Each procedure call creates a stack frame.

Heap:

- ✓ Stores dynamically allocated memory.
- ✓ In C, memory is allocated using malloc() and in C++ /Java, memory is allocated using new.
- ✓ Heap data can be allocated and freed in any order.
- ✓ Grows upward fro the bottom of the dynamic data segment and managed by the memory allocator.

- **Global Data Segment :**
  - ✓ Used by the operating system.
  - ✓ Not directly accessible by user programs.
  - ✓ Typically used for: OS kernel, system level services and exception handling

KTU
ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

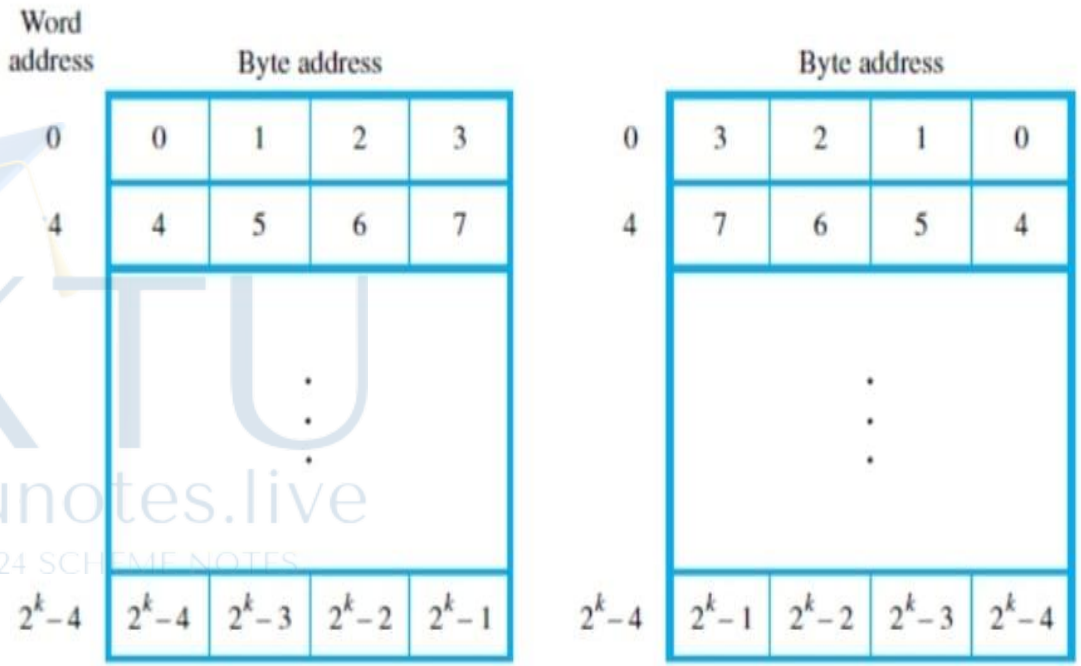
Byte Addressability

- A byte is always 8 bits, but word length ranges from 16 to 64 bits.
- Byte locations have addresses 0, 1, 2 …. Thus if the word length of the machine is 32 bits, successive words are located at addresses 0, 4, 8 …. With each word consisting of four bytes.
- It is of two types:

✓ **Big-Endian** : lower byte used for MSB

✓ **Little-Endian** : lower byte used for LSB

Note: In both cases addresses 0, 4 and 8 are used.



KTU
ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

CISC Vs RISC Architecture

Features	CISC (Complex Instruction Set)	RISC (Reduced Instruction Set)
Instruction set	Large and complex	Small and simple
Instruction length	Variable length	Fixed length
Instruction execution	Multiple clock cycles	Single clock cycle
Addressing modes	Many addressing modes	Few addressing modes
Memory access	Instruction can access memory directly	Only load/store instructions access memory
Registers	Fewer registers	Large number of registers
Control unit	Micro-programmed control	Hardwired control
Pipeline support	Difficult due to variable instructions	Easy and efficient
Hardware complexity	High	Simple
Compiler role	Less important	Very important
Code size	Smaller	Larger
Performance	Slower compared to RISC	Faster due to pipelining
Power consumption	Higher	Lower
Examples	Intel x86	RISC-V, ARM, MIPS

KTU ktunotes.live 2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE
Universal Engineering College, Irinjalakuda

Basic Instruction Types

- **Three Address Instruction**:

  Add A, B, C

  Source Operands : A and B

  Destination Operands : C

  Operation : Add

- **Two Address Instruction**:

  Add A, B add A and B and store the result in memory location B

  Move B, C move the content of B into location C

  Add A, C Add A and C and store the sum in location C

- **One Address Instruction**:

  A processor register called accumulator is used.

  Add A - add the content of memory location A to the contents of accumulator register and place the sum back to the accumulator.

  Load A - Load instruction copies the content of memory location A into accumulator.

  Store A - Store instruction copies the content of accumulator into memory location A

ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

Programming Concepts:

✓ **Arithmetic/Logical Instructions:** The various logical operations are given below.

	Source Registers							
$s1	1111	1111	1111	1111	0000	0000	0000	0000
$s2	0100	0110	1010	0001	1111	0000	1011	0111

Assembly Code		Result							
`and $s3, $s1, $s2`	$s3	0100	0110	1010	0001	0000	0000	0000	0000
`or $s4, $s1, $s2`	$s4	1111	1111	1111	1111	1111	0000	1011	0111
`xor $s5, $s1, $s2`	$s5	1011	1001	0101	1110	1111	0000	1011	0111
`nor $s6, $s1, $s2`	$s6	0000	0000	0000	0000	0000	1111	0100	1000

The logical operations with immediate values is shown in fig below.

	Source Values							
$s1	0000	0000	0000	0000	0000	0000	1111	1111
imm	0000	0000	0000	0000	1111	1010	0011	0100

zero-extended

Assembly Code		Result							
`andi $s2, $s1, 0xFA34`	$s2	0000	0000	0000	0000	0000	0000	0011	0100
`ori $s3, $s1, 0xFA34`	$s3	0000	0000	0000	0000	1111	1010	1111	1111
`xori $s4, $s1, 0xFA34`	$s4	0000	0000	0000	0000	1111	1010	1100	1011

KTU
ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

✓ **Shift Instructions:** shifts the value in a register left or right by up to 31 bits. The shift instruction machine code is given below.

Assembly Code	Field Values						Machine Code						
	op	rs	rt	rd	shamt	funct	op	rs	rt	rd	shamt	funct	
sll $t0, $s1, 4	0	0	17	8	4	0	000000	00000	10001	01000	00100	000000	(0x00114100)
srl $s2, $s1, 4	0	0	17	18	4	2	000000	00000	10001	10010	00100	000010	(0x00119102)
sra $s3, $s1, 4	0	0	17	19	4	3	000000	00000	10001	10011	00100	000011	(0x00119903)
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

The shift operation is given as:

Source Values

$s1	1111	0011	0000	0000	0000	0010	1010	1000
shamt								00100

Assembly Code		Result							
sll $t0, $s1, 4	$t0	0011	0000	0000	0000	0010	1010	1000	0000
srl $s2, $s1, 4	$s2	0000	1111	0011	0000	0000	0000	0010	1010
sra $s3, $s1, 4	$s3	1111	1111	0011	0000	0000	0000	0010	1010

Downloaded from KTUNOTES.LIVE

Variable shift instruction machine code

Assembly Code	Field Values						Machine Code						
	op	rs	rt	rd	shamt	funct	op	rs	rt	rd	shamt	funct	
sllv $s3, $s1, $s2	0	18	17	19	0	4	000000	10010	10001	10011	00000	000100	(0x02519804)
srlv $s4, $s1, $s2	0	18	17	20	0	6	000000	10010	10001	10100	00000	000110	(0x0251A006)
srav $s5, $s1, $s2	0	18	17	21	0	7	000000	10010	10001	10101	00000	000111	(0x0251A807)
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Variable shift operation

Source Values

$s1	1111	0011	0000	0100	0000	0010	1010	1000
$s2	0000	0000	0000	0000	0000	0000	0000	**1000**

Assembly Code	Result								
sllv $s3, $s1, $s2	$s3	0000	0100	0000	0010	1010	1000	0000	0000
srlv $s4, $s1, $s2	$s4	0000	0000	1111	0011	0000	0100	0000	0010
srav $s5, $s1, $s2	$s5	1111	1111	1111	0011	0000	0100	0000	0010

Downloaded from KTUNOTES.LIVE

- ✓ **Multiplication & Division Instructions:**
- Multiply two 32-bit numbers produces a 64-bit product.
- Dividing two 32-bit numbers produces a 32-bit quotient and a 32-bit remainder.
- MIPS architecture uses two special purpose registers hi and lo which is used to hold the results of multiplication and division.

✓ **Branching:**

Branch instructions modify the program counter to skip over sections of code or to go back to repeat previous code. There are two types :

- ➢ Conditional Branches : using beq and bne
- ➢ Unconditional Branches : using j and jr

KTU
ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

- **Conditional branching using beq:**

MIPS Assembly Code

```
  addi  $s0, $0, 4          # $s0 = 0 + 4 = 4
  addi  $s1, $0, 1          # $s1 = 0 + 1 = 1
  sll   $s1, $s1, 2         # $s1 = 1 << 2 = 4
  beq   $s0, $s1, target    # $s0 == $s1, so branch is taken
  addi  $s1, $s1, 1         # not executed
  sub   $s1, $s1, $s0       # not executed

target:
  add   $s1, $s1, $s0       # $s1 = 4 + 4 = 8
```

- **Conditional branching using bne:**

MIPS Assembly Code

```
  addi   $s0, $0, 4          # $s0 = 0 + 4 = 4
  addi   $s1, $0, 1          # $s1 = 0 + 1 = 1
  sll    $s1, $s1, 2         # $s1 = 1 << 2 = 4
  bne    $s0, $s1, target    # $s0 == $s1, so branch is not taken
  addi   $s1, $s1, 1         # $s1 = 4 + 1 = 5
  sub    $s1, $s1, $s0       # $s1 = 5 - 4 = 1

target:
  add    $s1, $s1, $s0       # $s1 = 1 + 4 = 5
```

KTU
ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

- **Unconditional branching using j:**

MIPS Assembly Code

```
  addi   $s0, $0, 4      # $s0 = 4
  addi   $s1, $0, 1      # $s1 = 1
  j      target          # jump to target
  addi   $s1, $s1, 1     # not executed
  sub    $s1, $s1, $s0   # not executed
target:
  add    $s1, $s1, $s0   # $s1 = 1 + 4 = 5
```

- **Unconditional branching using jr:**

MIPS Assembly Code

```
0x00002000   addi   $s0, $0, 0x2010   # $s0 = 0x2010
0x00002004   jr     $s0               # jump to 0x00002010
0x00002008   addi   $s1, $0, 1        # not executed
0x0000200c   sra    $s1, $s1, 2       # not executed
0x00002010   lw     $s3, 44 ($s1)     # executed after jr instruction
```

ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

✓ **Conditional Statements:** These are commonly used by high level languages. They execute a block of code consisting of one or more instructions.

Code Example 6.16 `if` STATEMENT

High-Level Code

```
if (i == j)
  f = g + h;

f = f - i;
```

MIPS Assembly Code

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
  bne $s3, $s4, L1      # if i ! = j, skip if block
  add $s0, $s1, $s2     # if block: f = g + h
L1:
  sub $s0, $s0, $s3     # f = f - i
```

Code Example 6.17 `if/else` STATEMENT

High-Level Code

```
if (i == j)
  f = g + h;

else
  f = f - i;
```

MIPS Assembly Code

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
  bne $s3, $s4, else      # if i ! = j, branch to else
  add $s0, $s1, $s2       # if block: f = g + h
  j   L2                  # skip past the else block
else:
  sub $s0, $s0, $s3       # else block: f = f - i
L2:
```

Downloaded from KTUNOTES.LIVE

Code Example 6.18 switch/case STATEMENT

High-Level Code

```
switch (amount) {
  case 20: fee = 2; break;

  case 50: fee = 3; break;

  case 100: fee = 5; break;

  default: fee = 0;
}

// equivalent function using if/else statements
  if      (amount == 20)  fee = 2;
  else if (amount == 50)  fee = 3;
  else if (amount == 100) fee = 5;
  else                    fee = 0;
```

MIPS Assembly Code

```
# $s0 = amount, $s1 = fee
case20:
  addi $t0, $0,  20      # $t0 = 20
  bne  $s0, $t0, case50  # i == 20? if not,
                         #   skip to case50
  addi $s1, $0,  2       # if so, fee = 2
  j    done              # and break out of case
case50:
  addi $t0, $0,  50      # $t0 = 50
  bne  $s0, $t0, case100 # i == 50? if not,
                         #   skip to case100
  addi $s1, $0,  3       # if so, fee = 3
  j    done              # and break out of case
case100:
  addi $t0, $0,  100     # $t0 = 100
  bne  $s0, $t0, default # i == 100? if not,
                         #   skip to default
  addi $s1, $0,  5       # if so, fee = 5
  j    done              # and break out of case
default:
  add  $s1, $0, $0       # charge = 0

done:
```

KTU ktunotes.live 2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

✓ **Loops:** Executes a block of code depending on the condition.

Code Example 6.19 while LOOP

High-Level Code

```
int pow = 1;
int x   = 0;



while (pow != 128)
{
  pow = pow * 2;
  x = x + 1;
}
```

MIPS Assembly Code

```
# $s0 = pow, $s1 = x
  addi  $s0, $0, 1       # pow = 1
  addi  $s1, $0, 0       # x = 0

  addi  $t0, $0, 128     # t0 = 128 for comparison
while:
  beq   $s0, $t0, done   # if pow == 128, exit while
  sll   $s0, $s0, 1      # pow = pow * 2
  addi  $s1, $s1, 1      # x = x + 1
  j     while
done:
```

Code Example 6.20 for LOOP

High-Level Code

```
int sum = 0;



for (i = 0; i ! = 10; i = i + 1) {
  sum = sum + i;



}

// equivalent to the following while loop
int sum = 0;
int i = 0;
while (i != 10) {
  sum = sum + i;
  i = i + 1;
}
```

MIPS Assembly Code

```
# $s0 = i, $s1 = sum
  add   $s1, $0, $0      # sum = 0
  addi  $s0, $0, 0       # i   = 0
  addi  $t0, $0, 10      # $t0 = 10
for:
  beq   $s0, $t0, done   # if i == 10, branch to done
  add   $s1, $s1, $s0    # sum = sum + i
  addi  $s0, $s0, 1      # increment i
  j     for
done:
```

KTU ktunotes.live

Downloaded from KTUNOTES.LIVE

Example 6.6 LOOPS USING `slt`

The following high-level code adds the powers of 2 from 1 to 100. Translate it into assembly language.

```
// high-level code

int sum = 0;
for (i = 1; i < 101; i = i * 2)
  sum = sum + i;
```

Solution: The assembly language code uses the set less than (`slt`) instruction to perform the less than comparison in the `for` loop.

```
# MIPS assembly code

# $s0 = i, $s1 = sum
  addi $s1, $0, 0    # sum = 0
  addi $s0, $0, 1    # i = 1
  addi $t0, $0, 101  # $t0 = 101

loop:
  slt $t1, $s0, $t0     # if (i < 101) $t1 = 1, else $t1 = 0
  beq $t1, $0, done     # if $t1 == 0 (i >= 101), branch to done
  add $s1, $s1, $s0     # sum = sum + i
  sll $s0, $s0, 1       # i = i * 2
  j   loop

done:
```

KTU
ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

✓ **Arrays:**

- It is useful for accessing large amount of data.
- Each array element is identified by a number called its index.
- The number of elements in the array is called the size of the array.

Array Indexing:

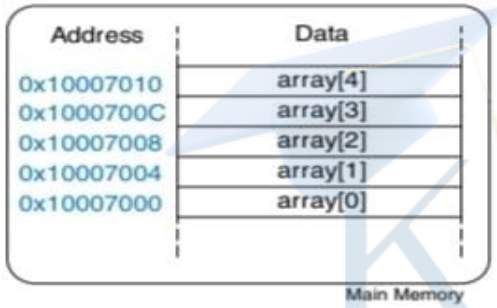


Figure 6.20 Five-entry array with base address of 0x10007000

Code Example 6.21 ACCESSING ARRAYS

High-Level Code

```
int array [5];

array[0] = array[0] * 8;

array[1] = array[1] * 8;
```

MIPS Assembly Code

```
# $s0 = base address of array
  lui   $s0, 0x1000       # $s0 = 0x10000000
  ori   $s0, $s0, 0x7000  # $s0 = 0x10007000

  lw    $t1, 0($s0)       # $t1 = array[0]
  sll   $t1, $t1, 3       # $t1 = $t1 << 3 = $t1 * 8
  sw    $t1, 0($s0)       # array[0] = $t1

  lw    $t1, 4($s0)       # $t1 = array[1]
  sll   $t1, $t1, 3       # $t1 = $t1 << 3 = $t1 * 8
  sw    $t1, 4($s0)       # array[1] = $t1
```

KTU
ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

✓ **Arrays:**

- It is useful for accessing large amount of data.
- Each array element is identified by a number called its index.
- The number of elements in the array is called the size of the array.

Array Indexing:

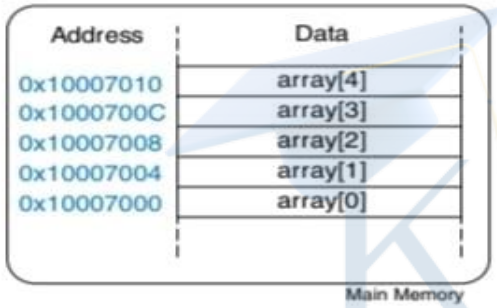


Figure 6.20 Five-entry array with base address of 0x10007000

Code Example 6.21 ACCESSING ARRAYS

High-Level Code

```
int array [5];

array[0] = array[0] * 8;

array[1] = array[1] * 8;
```

MIPS Assembly Code

```
# $s0 = base address of array
  lui   $s0, 0x1000       # $s0 = 0x10000000
  ori   $s0, $s0, 0x7000  # $s0 = 0x10007000

  lw    $t1, 0($s0)       # $t1 = array[0]
  sll   $t1, $t1, 3       # $t1 = $t1 << 3 = $t1 * 8
  sw    $t1, 0($s0)       # array[0] = $t1

  lw    $t1, 4($s0)       # $t1 = array[1]
  sll   $t1, $t1, 3       # $t1 = $t1 << 3 = $t1 * 8
  sw    $t1, 4($s0)       # array[1] = $t1
```

KTU
ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

Code Example 6.22 ACCESSING ARRAYS USING A `for` LOOP

High-Level Code

```
int i;
int array[1000];

for (i=0; i < 1000; i = i + 1) {

  array[i] = array[i] * 8;

}
```

MIPS Assembly Code

```
# $s0 = array base address, $s1 = i
# initialization code
  lui   $s0, 0x23B8       # $s0 = 0x23B80000
  ori   $s0, $s0, 0xF000  # $s0 = 0x23B8F000
  addi  $s1, $0           # i = 0
  addi  $t2, $0, 1000     # $t2 = 1000

loop:
  slt   $t0, $s1, $t2     # i < 1000?
  beq   $t0, $0, done     # if not then done
  sll   $t0, $s1, 2       # $t0 = i * 4 (byte offset)
  add   $t0, $t0, $s0     # address of array[i]
  lw    $t1, 0($t0)       # $t1 = array[i]
  sll   $t1, $t1, 3       # $t1 = array[i] * 8
  sw    $t1, 0($t0)       # array[i] = array[i] * 8
  addi  $s1, $s1, 1       # i = i + 1
  j     loop              # repeat
done:
```

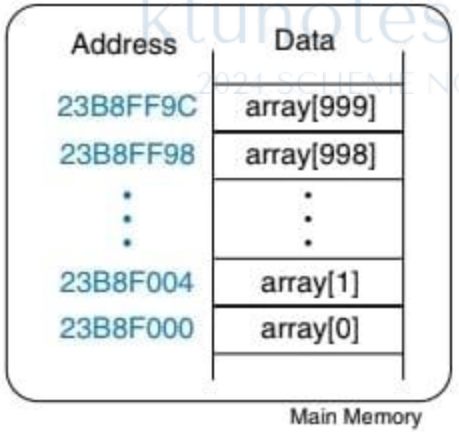


Figure 6.21 **Memory holding array[1000] starting at base address 0x23B8F000**

Downloaded from KTUNOTES.LIVE

Addressing Modes:

- **Register-only addressing**: It uses registers for all source and destination operands. All R-type instructions use register-only addressing.
- **Immediate addressing**: It uses 16-bit immediate along with registers as operands. Some I-type instructions, such as add immediate (addi) and load upper immediate (lwi), use immediate addressing.
- **Byte addressing**: Memory access instructions such as lw and sw use base addressing.

  Effective address of the memory operand = base address in register rs + sign extended bit offset found in immediate field.
- PC-relative addressing:

✓ Conditional branch instructions use PC relative addressing to specify the new value of the PC if the branch is taken.

✓ The signed offset in the immediate field is added to the PC to obtain the new PC

✓ Hence branch destination address is said to be relative to the current PC.

Downloaded from KTUNOTES.LIVE

Code Example 6.28 CALCULATING THE BRANCH TARGET ADDRESS

MIPS Assembly Code

```
0xA4          beq  $t0, $0, else
0xA8          addi $v0, $0, 1
0xAC          addi $sp, $sp, 8
0xB0          jr   $ra
0xB4    else: addi $a0, $a0, -1
0xB8          jal  factorial
```

Assembly Code	Field Values				Machine Code				
	op	rs	rt	imm	op	rs	rt	imm	
`beq $t0, $0, else`	4	8	0	3	000100	01000	00000	0000 0000 0000 0011	(0x11000003)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits	

Figure 6.28 **Machine code for** `beq`

Example 6.8 CALCULATING THE IMMEDIATE FIELD FOR PC-RELATIVE ADDRESSING

Calculate the immediate field and show the machine code for the branch not equal (`bne`) instruction in the following program.

```
# MIPS assembly code
0x40 loop: add  $t1, $a0, $s0
0x44       lb   $t1, 0($t1)
0x48       add  $t2, $a1, $s0
0x4C       sb   $t1, 0($t2)
0x50       addi $s0, $s0, 1
0x54       bne  $t1, $0, loop
0x58       lw   $s0, 0($sp)
```

Solution: Figure 6.29 shows the machine code for the `bne` instruction. Its branch target address, 0x40, is 6 instructions behind `PC`+4 (0x58), so the immediate field is −6.

Assembly Code	Field Values				Machine Code				
	op	rs	rt	imm	op	rs	rt	imm	
`bne $t1, $0, loop`	5	9	0	-6	000101	01001	00000	1111 1111 1111 1010	(0x1520FFFA)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits	

ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

- **Pseudo-Direct addressing**:

- ✓ In direct addressing, an address is specified in the instruction.
- ✓ The jump instructions, j and jal ideally would use direct addressing to specify a 32-bit jump target address (JTA) to indicate the instruction address to execute next.

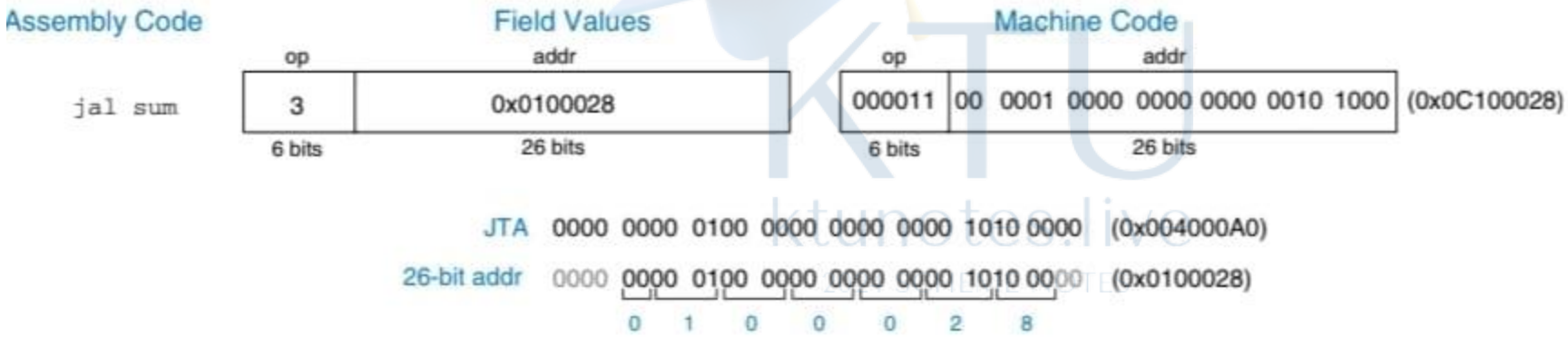


Figure 6.30 `jal` **machine code**

Downloaded from KTUNOTES.LIVE

Machine Language:

- The low level programming language consisting of binary 0s and 1s instructions directly executed by the CPU.
- The different types of machine language instructions are:

✓ **R-Type Instructions: (Register Type)**

The frame format for R-type instruction is as follows:

R-type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

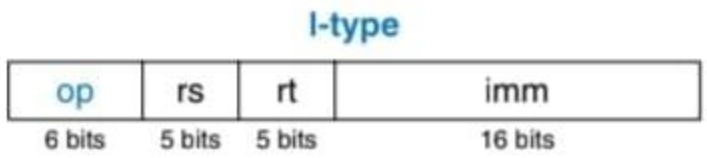
The machine code for R-type instruction is:

Assembly Code	Field Values						Machine Code						
	op	rs	rt	rd	shamt	funct	op	rs	rt	rd	shamt	funct	
add $s0, $s1, $s2	0	17	18	16	0	32	000000	10001	10010	10000	00000	100000	(0x02328020)
sub $t0, $t3, $t5	0	11	13	8	0	34	000000	01011	01101	01000	00000	100010	(0x016D4022)
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Downloaded from KTUNOTES.LIVE

✓ I-Type Instructions: (Immediate type)

The frame format for I-type instruction is as follows:



The machine code for I-type instruction is:

Assembly Code	Field Values op	rs	rt	imm	Machine Code op	rs	rt	imm	
`addi $s0, $s1, 5`	8	17	16	5	001000	10001	10000	0000 0000 0000 0101	(0x22300005)
`addi $t0, $s3, -12`	8	19	8	–12	001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
`lw $t2, 32($0)`	35	0	10	32	100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
`sw $s1, 4($t1)`	43	9	17	4	101011	01001	10001	0000 0000 0000 0100	(0xAD310004)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits	

KTU
ktunotes.live
2024 SCHEME NOTES

✓ J-Type Instruction : (Jump type)

It is used only with jump instructions and uses 26-bit address operand & 6-bit opcode.

Downloaded from KTUNOTES.LIVE

THANK YOU

ktunotes.live
2024 SCHEME NOTES

Downloaded from KTUNOTES.LIVE

JOIN OUR WHATSAPP GROUP FOR UPDATEDS

WHATSAPP GROUP

VISIT OUR WEBSITE FOR STUDY MATERIALS

WEBSITE